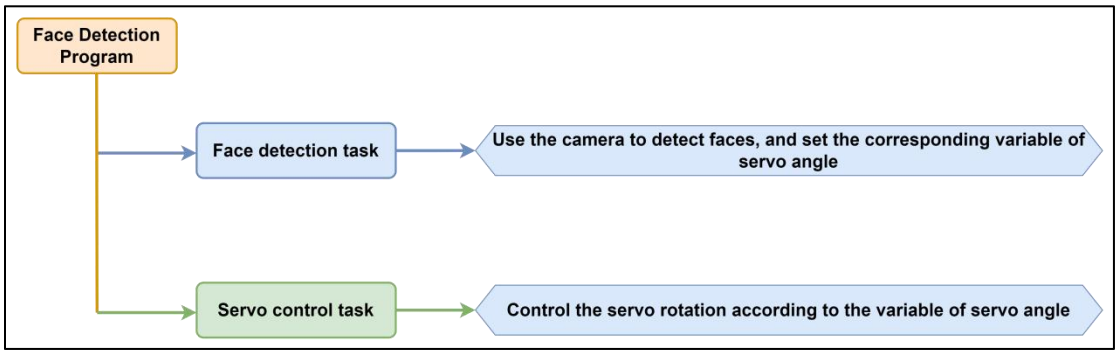
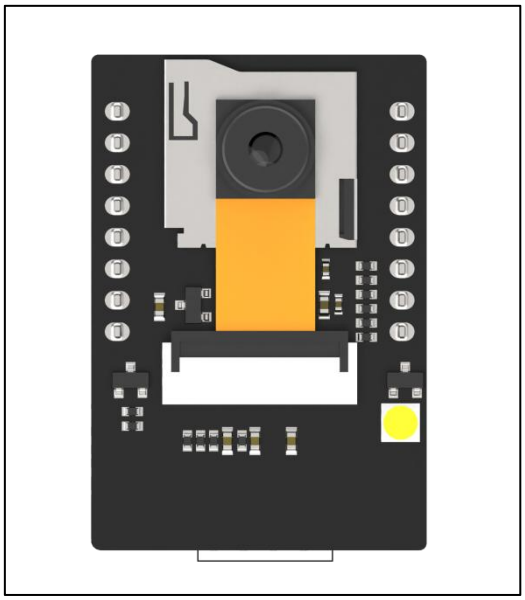
6.6 Face Detection

In this section, MiniArm will use ESP32-Cam to recognize faces, then perform a waving gesture after a face is detected.

1. Program Flowchart



2. ESP32-Cam Development Board



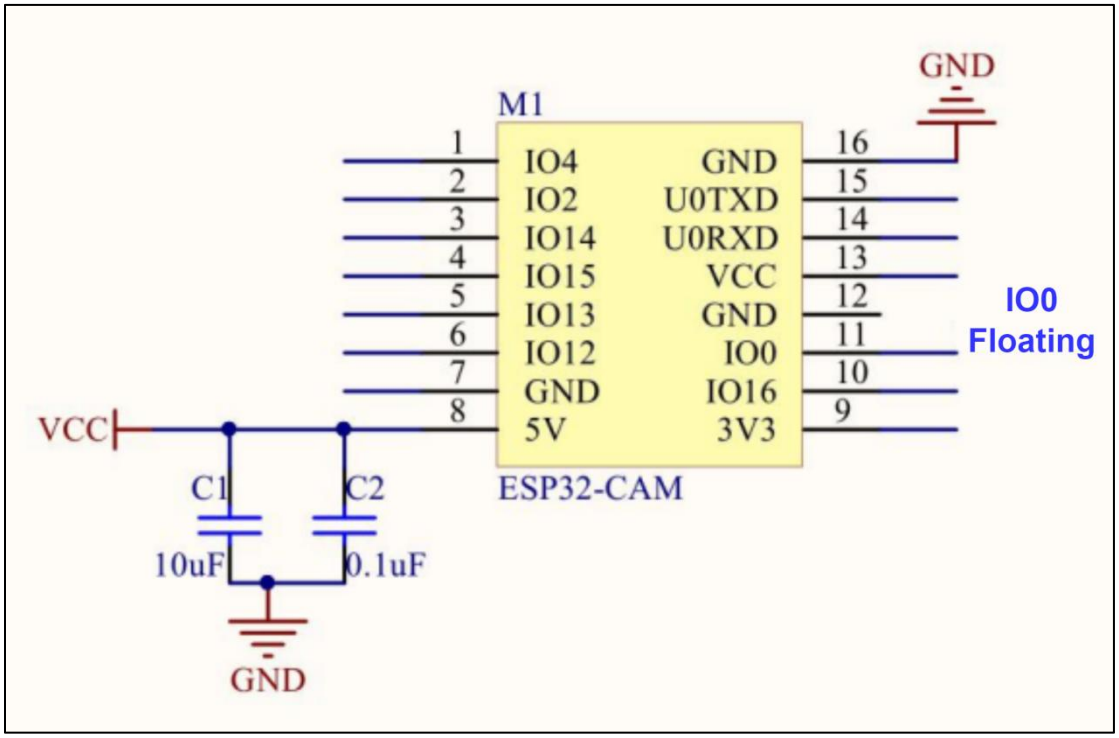
ESP32-Cam development board integrates an ESP32 chip and a camera module. After being inserted into the expansion board, it uses the I2C communication interface to read color and face data through I2C communication.

2.1 Working Principle

ESP32-CAM is equipped with an OV2640 camera module, which transmits the captured image data to the ESP32 chip via a serial port. The ESP32 chip can process the image and transmit it to other devices through I2C communication.

2.2 Schematic Diagram

ESP32-CAM is controlled by the ESP32 chip and is responsible for the control and communication functions of the entire system. The module captures the image data through the OV2640 camera, processes and analyzes it, and finally executes corresponding operations.



Supply Voltage: 5V

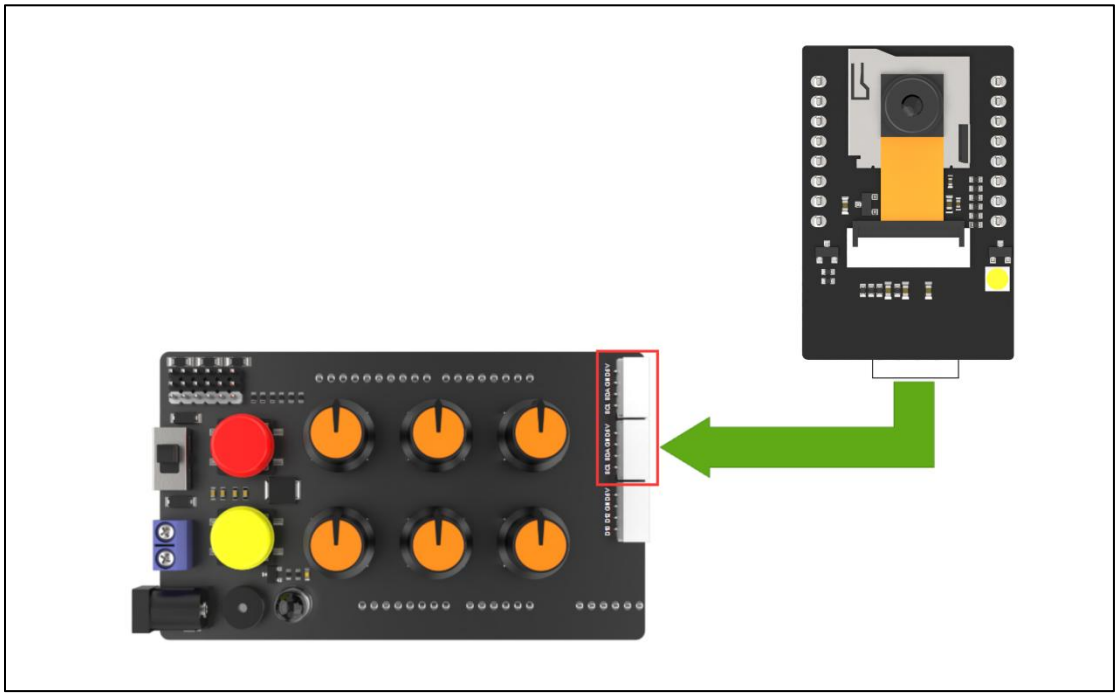
Power Consumption: 6mA-180mA

Working Temperature: -20˚C~70˚C

2.3 Wiring

Align the pins of ESP32-Cam with the pin headers on the camera

expansion board, and connect it to an I2C interface on the MiniArm expansion board with a 4PIN wire.



3. Program Download

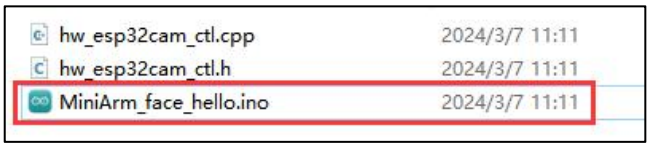
Note:

◆ Remove the Bluetooth module before downloading the program. Or the program will fail to download because of the serial port conflict.

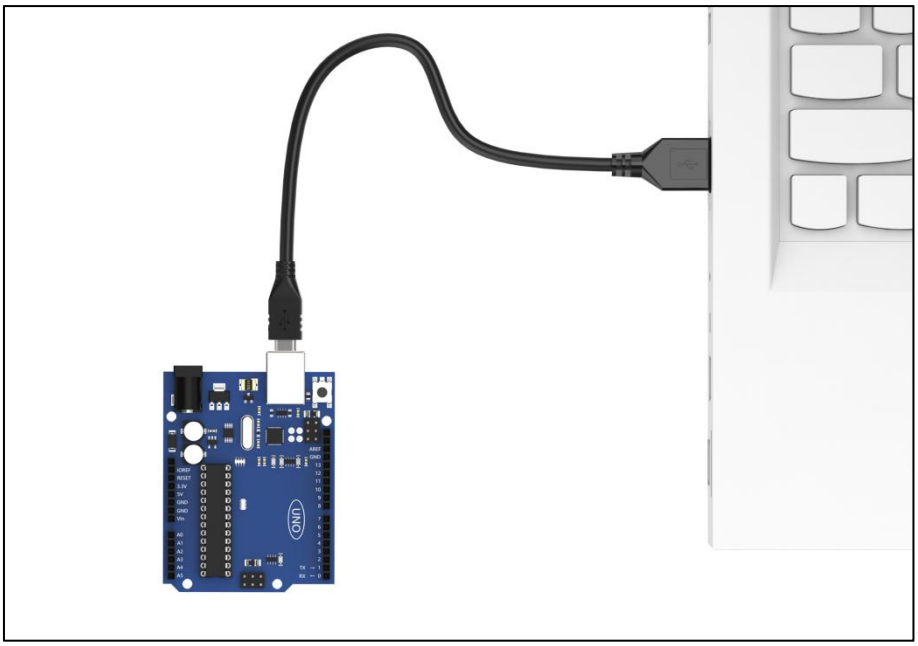
◆ Please switch the battery box to the “OFF” when connecting the Type-B download cable. This action prevents the download cable from accidentally touching the power pins of the expansion board, which may cause a short circuit.

3.1 Arduino UNO Program Download

1) Locate and open the “MiniArm_face_hello\MiniArm_face_hello.ino” program file in the same directory as this section.



2) Connect Arduino to the computer with the UNO cable (Type-B).



3) Click “Select Development Board”, and the software will automatically detect the current Arduino serial port. Next, click to connect.

4) Click to download the program into Arduino. Then just wait for it to complete.

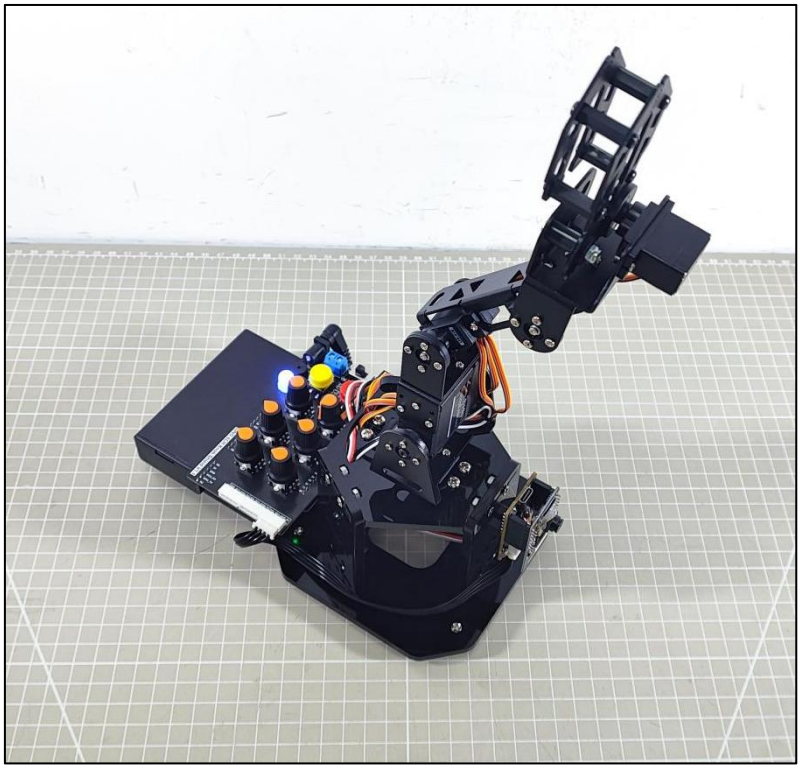
3.2 ESP32-Cam Firmware Flashing

1) According to the “1. Tutorials\6. AI Vision Game Course\6.3 Live Camera Feed-6.1 Live Camera Feed Firmware Flashing”, flash the “ESP32Cam_face_detect_bin\face_IIC_slave_vf0.bin” firmware in the same path of this document to ESP32-Cam.

2) After the flashing is completed, disconnect the Type-C cable and turn on the robotic arm.

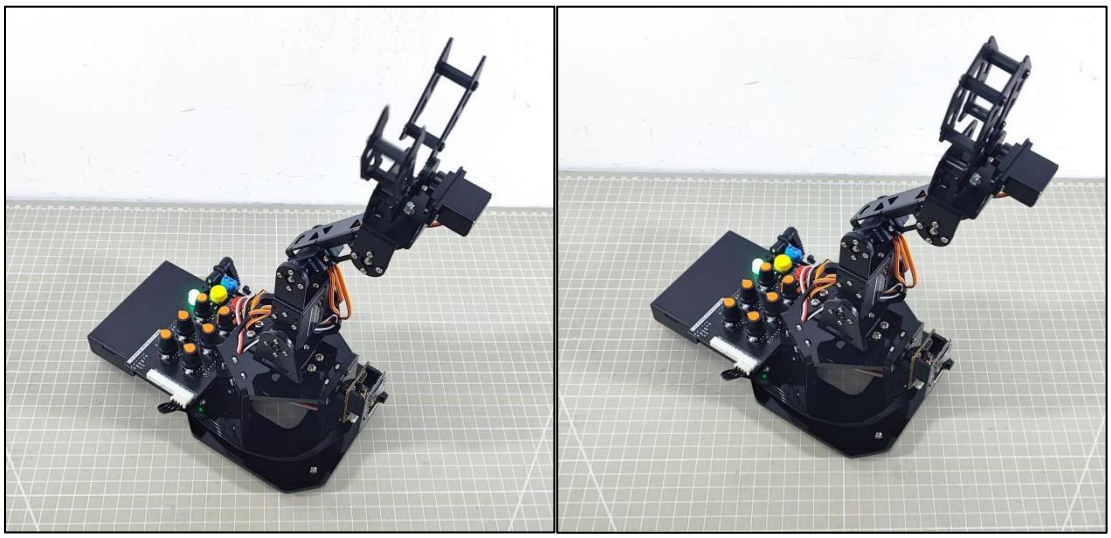
4. Program Outcome

1) After being powered on, the robotic arm returns to the neutral position, and the RGB light on the expansion board lights up blue.



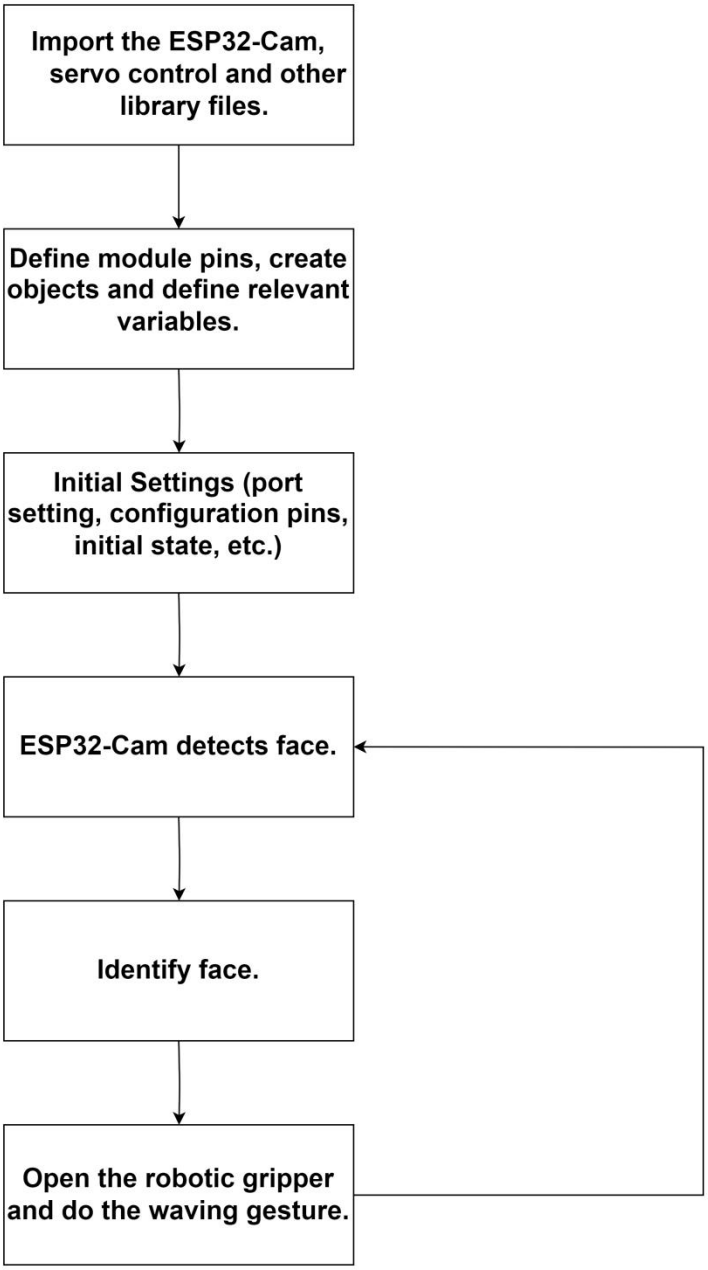
2) When a face is detected, the robotic gripper opens and closes, making

a waving action.



5. Brief Program Analysis

The name of this program is “MiniArm_face_hello.ino”. You can refer to the following flowchart for the implementation logic of the program:



5.1 Import Library File

1) Import the required libraries for RGB control, servo control, and the ESP32Cam communication library files for the game.

```
#include <FastLED.h> //import LED library
#include <Servo.h> //import servo library
#include "hw_esp32cam_ctl.h" //import ESP32Cam communication library
#include "tone.h" //tone library
```

5.2 Define Pin and Create Object

1) Firstly, define Arduino pins used for connecting hardware, mainly consisting of five servo pins, one buzzer pin and one RGB light pin.

```
const static uint8_t servoPins[5] = { 7, 6, 5, 4, 3 };
const static uint8_t buzzerPin = 11;
const static uint8_t rgbPin = 13;
```

2) Then, define the variables for RGB light control, ESP32Cam communication, and servos. The array “extended_func_angles” is used to store each servo’s desired angle. The “limt_angles” array is used to limit the servo’s rotation angle. The “servo_angles” array is used to store each servo’s actual angle.

```
//RGB light control object
static CRGB rgbs[1];
//ESP32Cam communication object
HW_ESP32Cam hw_cam;
//servo control object
Servo servos[5];

/* the used angle values of secondary development example */
static uint8_t extended_func_angles[5] = { 90, 140, 140, 87, 88 };
/* limit of each joint’s angle */
const uint8_t limt_angles[5][2] = {{0,92},{0,180},{0,180},{25,180},{0,180} };
/* Actual angle values of servo control */
static uint8_t servo_angles[5] = { 90, 140, 140, 87, 88 };
```

Regarding the desired and actual values, differences may arise between them during the servo rotation control process due to various factors. You can inspect the “servo_control” function called within the main “loop” function for

further details.

If you want the servo to gradually move towards the target position, you need to call “servo_angles[i] = servo_angles[i] * 0.8 + extended _func_ a ngles[i] * 0.2” in the control task function.

This allows the servo to move each time at 80% of the actual value and 20% of the expected value, so that the actual value will gradually approach the expected value. And when it is equal to the expected value, the robotic arm stops moving.

```
void servo_control(void) {
  static uint32_t last_tick = 0;
  if (millis() - last_tick < 20) {
    return;
  }
  last_tick = millis();

  for (int i = 0; i < 5; ++i) {
    if(servo_angles[i] > extended_func_angles[i])
    {
      servo_angles[i] = servo_angles[i] * 0.8 + extended_func_angles[i] * 0.2;
      if(servo_angles[i] < extended_func_angles[i])
        servo_angles[i] = extended_func_angles[i];
    }else if(servo_angles[i] < extended_func_angles[i])
    {
      servo_angles[i] = servo_angles[i] * 0.8 + (extended_func_angles[i] * 0.2 + 1);
      if(servo_angles[i] > extended_func_angles[i])
        servo_angles[i] = extended_func_angles[i];
    }
    servo_angles[i] = servo_angles[i] < limt_angles[i][0] ? limt_angles[i][0] : servo_angles[i];
    servo_angles[i] = servo_angles[i] > limt_angles[i][1] ? limt_angles[i][1] : servo_angles[i];
    servos[i].write(i == 0 || i == 5 ? 180 - servo_angles[i] : servo_angles[i]);
  }
}
```

5.3 Initialization

1) The “setup ()” function mainly initializes associated hardware devices. Starting with the serial port, which sets the baud rate as “115200” and a read timeout as 500ms.

```
void setup() {
  // put your setup code here, to run once:
  Serial.begin(115200);
  // set the timeout for reading data from the serial port
  Serial.setTimeout(500);
```

2) Assign the servo IO port to facilitate the control through the pin.

```
for (int i = 0; i < 5; ++i) {
  servos[i].attach(servoPins[i]);
}
```

3) Initialize the communication connector of ESP32-CAM and set the fill light brightness to 0. Within the range of 0 to 255. If it is 0, close the fill light.

```
hw_cam.begin(); //initialize the ESP32Cam communication connector
hw_cam.set_led(10); // control ESP32Cam fill light brightness [brightness value (0~255)]
```

4) Use the “FastLED” library to initialize the RGB lights on the expansion board, and connect them to the RGB pins. Then set the color to blue by “rgbs[0] = CRGB(0, 0, 100)” and use the “FastLED.show” function to display the set color.

```
FastLED.addLeds<WS2812, rgbPin, GRB>(rgbs, 1);
rgbs[0] = CRGB(0, 0, 100);
FastLED.show();
```

5) Set the pin of the buzzer to output mode and stop activating the buzzer after a short sound.

```
pinMode(buzzerPin, OUTPUT);
tone(buzzerPin, 1000);
delay(100);
noTone(buzzerPin);
```

6) Output the “Start" string through a serial port to indicate that the sensor is ready to start transmitting data.

```
  delay(2000);
  Serial.println("start");
}
```

5.4 Call Subfunction in a Loop

After finishing the initialization, it enters the “loop” main function. First, call the “espcam_task” function which is used to detect face identification and perform the waving action. Then, call the “servo_control” function to control servos.

```
void loop() {
  // esp32cam communication task
  espcam_task();
  // servo control
  servo_control();
}
```

5.5 ESP32-Cam Communication Task

Define the “espcam_task” function to detect face recognition and perform the waving action.

1) Firstly, define 3 variables: “last_tick” records the timestamp of the last task execution; “step” tracks the current stage of task execution; and “delay_count” sets the counting unit for time delay.

```
void espcam_task(void)
{
  static uint32_t last_tick = 0;
  static uint8_t step = 0;
  static uint32_t delay_count = 0;
```

2) The “last_tick” variable and “millis()” are used for delay operation. Use the “millis()” function to obtain the running time of the program minus the “last_tick” variable. If the difference is less than 100, the function will return. If the difference is greater than or equal to 100, it means the task has been delayed by 50ms. Then the current time is assigned to the “last_tick” variable for the next delay.

```
if (millis() - last_tick < 50) {
  return;
}
last_tick = millis();
```

3) Assign the “rt” variable to the detection result.

```
bool rt = hw_cam.faceDetect();
```

4) Execute the waving action as soon as the face is detected. Stay stationary if the face is undetected. Here is the analysis of the “switch” code snippet:

```
switch(step)
{
  case 0:
    if(rt) //If a face is detected
    {
      delay_count = 0;
      rgbs[0].r = 0;
      rgbs[0].g = 100;
      rgbs[0].b = 0;
      FastLED.show();
```

◆ Case 0: detect face according to the “hw_cam.colorDetect()”. If a face is detected, illuminate the RGB light in red and proceed to the next state.

```
case 0:
  if(rt) //If a face is detected
  {
    delay_count = 0;
    rgbs[0].r = 0;
    rgbs[0].g = 100;
    rgbs[0].b = 0;
    FastLED.show();
    // adjust to the next step
    step++;
  }
  break;
```

◆ Case 1: use “extended_func_angles” to control the open of the robotic gripper and set ID1 servo angle to 82°. Wait for 300ms to enter the next state.

```
case 1: //open the gripper
  extended_func_angles[0] = 40;
  // wait for action completion
  delay_count++;
  if(delay_count > 6)
  {
    delay_count = 0;
    step++;
  }
  break;
```

◆ Case 2: close the robotic gripper and set ID1 servo angle to 82°. Wait for 300ms to enter the next state.

```
case 2: //close the gripper
  extended_func_angles[0] = 82;
  // wait for action completion
  delay_count++;
  if(delay_count > 6)
  {
    delay_count = 0;
    step++;
  }
  break;
```

◆ Case 3 and case 4 are the same as above.

```
case 3: //open the gripper
  extended_func_angles[0] = 40;
  // wait for action completion
  delay_count++;
  if(delay_count > 6)
  {
    delay_count = 0;
    step++;
  }
  break;
case 4: //close the gripper
  extended_func_angles[0] = 82;
  // wait for action completion
  delay_count++;
  if(delay_count > 10)
  {
    rgbs[0].r = 0;
    rgbs[0].g = 0;
    rgbs[0].b = 100;
    FastLED.show();
    delay_count = 0;
    step++;
  }
  break;
```

◆ After the action execution is completed, set the state value to 0 to restart the detecting.

```
      default:
        step = 0;
        break;
    }
  }
```

5.5 Servo Control Task

According to the target angle in the “extended_func_angles” array, the current angle in the “servo_angles” array, and each servo’s angle limit range, calculate and set each servo’s angle.

```
void servo_control(void) {
  static uint32_t last_tick = 0;
  if (millis() - last_tick < 20) {
    return;
  }
  last_tick = millis();

  for (int i = 0; i < 5; ++i) {
    if(servo_angles[i] > extended_func_angles[i])
```

1) Firstly, check the time interval since the last execution. If the interval is less than 20ms, the function will return immediately to avoid updating the servo angle too frequently.

```
void servo_control(void) {
  static uint32_t last_tick = 0;
  if (millis() - last_tick < 20) {
    return;
  }
  last_tick = millis();
```

2) By checking the difference between the “servo_angles[i]” (current servo angles) and the “extended_func_angles[i]” (target angles), adjust the servos’ rotation angles smoothly.

```
  for (int i = 0; i < 5; ++i) {
    if(servo_angles[i] > extended_func_angles[i])
    {
      servo_angles[i] = servo_angles[i] * 0.8 + extended_func_angles[i] * 0.2;
      if(servo_angles[i] < extended_func_angles[i])
        servo_angles[i] = extended_func_angles[i];
    }else if(servo_angles[i] < extended_func_angles[i])
    {
      servo_angles[i] = servo_angles[i] * 0.8 + (extended_func_angles[i] * 0.2 + 1);
      if(servo_angles[i] > extended_func_angles[i])
        servo_angles[i] = extended_func_angles[i];
    }
```

◆ Check if the current servo angles are greater than the target angles. If they are, smoothly adjust the angles by taking small steps toward the target angles (each step is 80% of the current angles plus 20% of the target angles).

```
  for (int i = 0; i < 5; ++i) {
    if(servo_angles[i] > extended_func_angles[i])
    {
      servo_angles[i] = servo_angles[i] * 0.8 + extended_func_angles[i] * 0.2;
```

◆ If the current servo angles are still greater than the target angles,

directly set them to the target angles.

```
if(servo_angles[i] < extended_func_angles[i])
  servo_angles[i] = extended_func_angles[i];
```

◆ Check if the current servo angles are less than the target angles. If they are, smoothly adjust the angles by taking small steps toward the target angles (each step is 80% of the current angles plus 20% of the target angles plus 1).

```
}else if(servo_angles[i] < extended_func_angles[i])
{
  servo_angles[i] = servo_angles[i] * 0.8 + (extended_func_angles[i] * 0.2 + 1);
```

◆ If the current servo angles are still less than the target angles, directly set them to the target angles.

```
  if(servo_angles[i] > extended_func_angles[i])
    servo_angles[i] = extended_func_angles[i];
}
```

3) After adjusting the angles, make sure that the current angles of the servo do not exceed its physical limits.

```
servo_angles[i] = servo_angles[i] < limt_angles[i][0] ? limt_angles[i][0] : servo_angles[i];
servo_angles[i] = servo_angles[i] > limt_angles[i][1] ? limt_angles[i][1] : servo_angles[i];
```

4) Write the calculated angles into the servo.

```
  servos[i].write(i == 0 || i == 5 ? 180 - servo_angles[i] : servo_angles[i]);
}
}
```

The method “servos[i].write()” is used to set the servo angles. It takes an angle value as a parameter and determines whether to invert it based on the value of “i”.

6. FAQ

Q: After the code is uploaded, the face cannot be recognized. What should I do?

A: Please check if the 4PIN wire is correctly connected to the I2C interface.

Q: Why the camera cannot recognize when I face it, but it can recognize when flipped upside down?

A: Please flash another bin file to enable recognition when facing forward.